Real-Time DevOps Tasks — Complete Practical Notes

Task Example 1 — New Version Deploy Cheyyali
Developer:
“Netflix application ki v2.3.0 release ready. Production lo deploy cheyyali.”
Company repository lo already deployment.yaml untundi. Existing image v2.2.0 undachu. Developer/build pipeline already new Docker image mycompany/netflix:v2.3.0 ni registry ki push chesi undachu. Nee task existing Deployment lo new image version use chesi production lo safely deploy cheyyadam. Deploy chesina tarvata new Pods successful ga create ayyaya, old Pods terminate ayyaya, application healthy ga unda ani verify cheyyali.
Existing deployment.yaml

```
apiVersion: apps/v1
kind: Deployment

metadata:
 name: netflix

spec:
 replicas: 3

 selector:
  matchLabels:
   app: netflix

 template:
  metadata:
   labels:
    app: netflix

  spec:
   containers:
    - name: netflix
      image: mycompany/netflix:v2.2.0
      ports:
       - containerPort: 80
```

Step 1 — Existing Deployment check
kubectl get deployment netflix
Example:

```
NAME     READY   UP-TO-DATE   AVAILABLE   AGE
netflix  3/3     3            3           20d
```

Ikkada 3/3 ante desired 3 Pods lo 3 ready ga unnayi.
Detailed ga:
kubectl describe deployment netflix
Ikkada **Image, Replicas, Selector, Events, ReplicaSet** details chudachu.
Step 2 — Current Pods check
kubectl get pods -l app=netflix
Example:

```
NAME                 READY   STATUS    RESTARTS
netflix-7c8d9f6b7d-abc12   1/1     Running   0
netflix-7c8d9f6b7d-def34   1/1     Running   0
netflix-7c8d9f6b7d-ghi56   1/1     Running   0
```

Ivi old version Pods ani image check cheyyali ante:
kubectl describe pod netflix-7c8d9f6b7d-abc12
or:
kubectl get deployment netflix -o yaml
Step 3 — New image set cheyyadam
Manual ga YAML edit cheyyachu:

```
image: mycompany/netflix:v2.3.0
```

Leda production lo quick image update kosam:
kubectl set image deployment/netflix netflix=mycompany/netflix:v2.3.0
Idi Deployment lo netflix container image ni v2.3.0 ki update chestundi.
Step 4 — Rollout status check
kubectl rollout status deployment/netflix
Successful ayithe:

```
deployment "netflix" successfully rolled out
```

Step 5 — New Pods check
kubectl get pods -l app=netflix
New ReplicaSet nunchi Pods create ayyi Running / Ready ga undali.
Problem vachindante?
Suppose:

```
netflix-xxxxx   0/1   ImagePullBackOff
```

Appudu new image Kubernetes pull cheyyalekapothundi.

Check:
kubectl describe pod netflix-xxxxx
Bottom lo **Events** section chudu.
Example:
Failed to pull image "mycompany/netflix:v2.3.0":
manifest unknown
Meaning v2.3.0 image/tag registry lo available lekapovachu or image name/tag wrong.
Check image manually:
docker pull mycompany/netflix:v2.3.0
Image available lekapothe developer/build team image correct ga build & push cheyyali.
Image correct ga unte private registry authentication issue undochu. Appudu imagePullSecrets configuration check cheyyali.
Rollout fail ayithe
Previous version ki rollback:
kubectl rollout undo deployment/netflix
Verify:
kubectl rollout status deployment/netflix
So complete flow:
Developer releases v2.3.0
↓
Docker image v2.3.0
↓
Registry
↓
DevOps updates Deployment
↓
Kubernetes creates new Pods
↓
New Pods Ready
↓
Old Pods removed
↓
Application verified

Task Example 2 — 3 Pods nunchi 6 Pods ki Increase Cheyyali
Production traffic perigindi anuko. Developer/manager:
“Netflix production replicas 3 nunchi 6 ki increase cheyyandi.”
Existing Deployment lo:
replicas: 3
untundi. Nee task 6 ki change cheyyadam. Important ga YAML change chesina tarvata actual Kubernetes state kuda verify cheyyali.
Simply file edit chesi vadileyakudadhu.
Step 1 — Current replicas check
kubectl get deployment netflix
Example:
NAME READY UP-TO-DATE AVAILABLE
netflix 3/3 3 3
Step 2 — Current Pods check
kubectl get pods -l app=netflix
3 Pods kanipistayi.
Step 3 — Scale 3 → 6
Fast method:
kubectl scale deployment netflix --replicas=6
Now Kubernetes desired replicas ni 6 ga change chestundi.
Step 4 — Deployment check
kubectl get deployment netflix
Initially:
READY 3/6
ila temporarily kanipinchachu.
Konchem time tarvata:
READY 6/6
Step 5 — Pods check
kubectl get pods -l app=netflix
Expected:
netflix-xxxxx 1/1 Running
netflix-yyyyy 1/1 Running
netflix-zzzzz 1/1 Running
netflix-aaaaa 1/1 Running
netflix-bbbbb 1/1 Running
netflix-ccccc 1/1 Running
YAML lo permanent ga maintain cheyyali ante
spec:
replicas: 6

Then:
kubectl apply -f deployment.yaml
Verify:
kubectl get deployment netflix
kubectl get pods -l app=netflix
New Pods create kakapothe?
Suppose:
READY 3/6
chala sepu same ga undi.
Appudu:
kubectl get pods -l app=netflix
Suppose:
netflix-new1 0/1 Pending
netflix-new2 0/1 Pending
netflix-new3 0/1 Pending
Pending ante Pod create ayindi kani node meeda schedule/start avvaledu.
Detailed reason:
kubectl describe pod netflix-new1
Bottom lo:
Events:
0/3 nodes are available:
Insufficient cpu
Ante cluster lo available CPU saripovatledu.
Appudu immediate ga replicas problem ani kaadu. **Cluster capacity problem**.
Node check:
kubectl get nodes
Node resources:
kubectl describe node <node-name>
Production lo appropriate solution additional capacity/nodes, resource requests tuning, or workload capacity planning avvachu.
Pod CrashLoopBackOff **ayithe?**
kubectl get pods -l app=netflix
Example:
netflix-new1 0/1 CrashLoopBackOff
Appudu:
kubectl logs netflix-new1
Application startup error kanipinchachu.
Previous crashed container logs kosam:
kubectl logs netflix-new1 --previous
So **3 → 6 scale command** simple:
kubectl scale deployment netflix --replicas=6
But real DevOps work lo important part command kante **6 Pods actually healthy ga unnaya ani verify cheyyadam**.

Task Example 3 — Application Down / 502 / 503 Errors
Developer:
“Netflix production site 502/503 errors istundi.”
Ikkada immediate ga YAML change cheyyakudadhu. First request flow ni reverse direction lo investigate cheyyali:
Internet
↓
Ingress / Load Balancer
↓
Service
↓
Pods
↓
Container
↓
Application
First problem exactly ekkada undho identify cheyyali.
Step 1 — Pods check
kubectl get pods -l app=netflix
Suppose:
netflix-abc123 0/1 CrashLoopBackOff
netflix-def456 1/1 Running
netflix-ghi789 1/1 Running
Ikkada immediate clue: abc123 unhealthy.
Step 2 — Pod logs
kubectl logs netflix-abc123
Suppose:
Error: Database connection failed
ECONNREFUSED
Ippudu application startup database ki connect avvalekapothundi ani identify chesam.

Step 3 — Pod details
kubectl describe pod netflix-abc123
Ikkada:

- container state
- restart count
- environment variables
- mounted ConfigMaps/Secrets
- image
- events

check cheyyachu.
Step 4 — Migatha Pods kuda check
kubectl get pods -l app=netflix
Only one Pod issue aa or all Pods issue aa identify cheyyali.
All 3 Pods:
CrashLoopBackOff
CrashLoopBackOff
CrashLoopBackOff
unte common configuration/database/image issue possibility ekkuva.
Only one Pod:
Running
Running
CrashLoopBackOff
unte individual Pod/node issue possibility untundi.
Step 5 — Service check
kubectl get svc netflix-service
Then:
kubectl describe svc netflix-service
Important ga Service ki backend endpoints unnaya check cheyyali.
kubectl get endpoints netflix-service
or modern clusters lo:
kubectl get endpointslice
Suppose endpoints empty ga unte:
Endpoints: <none>
Service ki matching healthy backend Pods levu ani clue.
Appudu Service selector:
kubectl describe svc netflix-service
and Pod labels:
kubectl get pods --show-labels
compare cheyyali.
Service:
selector:
app: netflix
Pods:
app=netflix
match avvali.
Step 6 — Ingress check
kubectl get ingress
Then:
kubectl describe ingress netflix-ingress
Ikkada:

- Host
- Path
- Backend Service
- Events

check cheyyali.
Suppose Ingress correct ga Service ki point chestundi, Service ki endpoints unnayi, Pods healthy ga unnayi. Appudu issue application-level or Ingress Controller/load-balancer side lo investigate cheyyali.
Step 7 — Application logs
kubectl logs -f <pod-name>
Live logs observe cheyyachu.
Suppose:
GET /api/movies 500
Database timeout
Appudu Kubernetes YAML problem kaakapovachu. Application/database dependency issue.
Real-time troubleshooting rule:
Error
↓
Check Pods

↓
Check Pod status
↓
Check logs
↓
Check describe/events
↓
Check Service
↓
Check endpoints
↓
Check Ingress
↓
Identify exact layer
↓
Fix
↓
Verify

Task Example 4 — Docker Image Pull Avvatledu
Deployment:
image: mycompany/netflix:v5
kani Pod:
ImagePullBackOff
ani chupistundi.
Ikkada container start avvaledu because node ki required image available/pull avvaledu.
Step 1 — Pod status
kubectl get pods
Example:
netflix-xxxxx 0/1 ImagePullBackOff
Step 2 — Exact reason
kubectl describe pod netflix-xxxxx
Bottom lo **Events** chudu.
Suppose:
Failed to pull image "mycompany/netflix:v5":
manifest unknown
Meaning registry lo v5 tag/image available lekapovachu.
Step 3 — Image name verify
Deployment:
kubectl get deployment netflix -o yaml
Search:
image: mycompany/netflix:v5
Check:

- repository name
- image name
- tag

correct aa?
Step 4 — Registry lo image exists aa check
If Docker registry access available:
docker pull mycompany/netflix:v5
Success ayithe image exists and credentials/environment batti further investigation cheyyali.
Fail:
manifest unknown
ante image/tag issue.
Developer/build pipeline correct image push cheyyali:
docker build -t mycompany/netflix:v5 .
docker push mycompany/netflix:v5
Then Kubernetes rollout restart/update:
kubectl rollout restart deployment/netflix
Verify:
kubectl rollout status deployment/netflix
kubectl get pods
Private registry ayithe
Events lo:
unauthorized
authentication required
unte registry authentication problem.
Deployment lo imagePullSecret configuration check cheyyali.
kubectl get secret
and:

kubectl describe deployment netflix
Private registry credentials correct ga configured unnaya check cheyyali.
So ImagePullBackOff chusi direct YAML edit cheyyadam kaadu. **Describe → Events → exact error → root cause → fix → verify**.

Task Example 5 — New Staging Environment Create Cheyyali
Company lo production already undi:
Netflix Production
replicas: 5
API_URL: production-api
Manager:
“Netflix ki staging environment create cheyyandi.”
Production configuration ni blindly copy chesi same values use cheyyakudadhu. Staging ki separate namespace, configuration and usually separate resources/environment-specific values maintain chestham.
Step 1 — Namespace create
namespace.yaml:

```
apiVersion: v1
kind: Namespace

metadata:
  name: netflix-staging
```

Apply:

```
kubectl apply -f namespace.yaml
```

Check:

```
kubectl get namespace netflix-staging
```

Step 2 — Staging Deployment

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: netflix
  namespace: netflix-staging

spec:
  replicas: 2

  selector:
    matchLabels:
      app: netflix

  template:
    metadata:
      labels:
        app: netflix

    spec:
      containers:
        - name: netflix
          image: mycompany/netflix:v2.3.0
          ports:
            - containerPort: 80
```

Apply:

```
kubectl apply -f deployment.yaml
```

Check specifically staging:

```
kubectl get pods -n netflix-staging
```

Step 3 — Staging configuration

```
apiVersion: v1
kind: ConfigMap

metadata:
  name: netflix-config
  namespace: netflix-staging

data:
  API_URL: "http://staging-api"
  APP_ENV: "staging"
```

Apply:

```
kubectl apply -f configmap.yaml
```

Step 4 — Staging Service

```
apiVersion: v1
kind: Service
```

```
metadata:
  name: netflix-service
  namespace: netflix-staging

spec:
  selector:
    app: netflix

  ports:
    - port: 80
      targetPort: 80

  type: ClusterIP
```

Apply:

```
kubectl apply -f service.yaml
```

Check:

```
kubectl get svc -n netflix-staging
```

Step 5 — Full staging verification

```
kubectl get all -n netflix-staging
```

This gives a quick overview of Pods, Services, Deployments etc.

Then:

```
kubectl get pods -n netflix-staging
kubectl get svc -n netflix-staging
kubectl describe deployment netflix -n netflix-staging
```

If Pod not ready:

```
kubectl describe pod <pod-name> -n netflix-staging
kubectl logs <pod-name> -n netflix-staging
```

So staging and production separate:

```
Cluster
│
├── netflix-prod
│     ├── Deployment
│     ├── Service
│     └── Pods
│
└── netflix-staging
      ├── Deployment
      ├── Service
      └── Pods
```

Task Example 6 — HPA / Automatic Scaling Improve Cheyyali

Company:

“Traffic periginaappudu Netflix Pods automatic ga scale avvali.”

Existing HPA:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler

metadata:
  name: netflix-hpa

spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: netflix

  minReplicas: 2
  maxReplicas: 10

  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

Meaning minimum 2 Pods, maximum 10 Pods, average CPU target around 70%.

Step 1 — HPA check

```
kubectl get hpa
```

Example:

```
NAME        REFERENCE          TARGETS   MINPODS   MAXPODS   REPLICAS
```

netflix-hpa Deployment/netflix 85%/70% 2 10 5
Ikkada:
85% / 70%
ante current CPU around 85%, target 70%.
HPA already 5 replicas ki scale chesindi.
Step 2 — Detailed HPA information
kubectl describe hpa netflix-hpa
Ikkada:

- current replicas
- desired replicas
- current CPU
- target CPU
- scaling events

chudachu.
Step 3 — Pods check
kubectl get pods -l app=netflix
Current replicas:
5 Pods
Step 4 — Requirement change
Suppose company requirement:
Minimum 3 Pods, maximum 20 Pods.
Then:
minReplicas: 3
maxReplicas: 20
Apply:
kubectl apply -f hpa.yaml
Verify:
kubectl get hpa netflix-hpa
HPA scale avvakapothe?
First:
kubectl describe hpa netflix-hpa
Events check cheyyali.
Then:
kubectl get pods
and:
kubectl top pods
kubectl top metrics unavailable ani error iste cluster metrics setup issue undochu.
Also Deployment lo CPU requests properly configured unnaya check cheyyali:
kubectl describe deployment netflix
Example:
resources:
requests:
cpu: "250m"
HPA CPU utilization calculation ki resource requests important.
So HPA issue ante:
HPA
↓
describe HPA
↓
metrics available aa?
↓
CPU usage visible aa?
↓
Pod resource requests unnaya?
↓
HPA target correct aa?
↓
Desired replicas change avtunnaya?
↓
Pods actually scale avtunnaya?

Task Example 7 — Pod Restart / CrashLoopBackOff Fix Cheyyali
Idi kuda very common.
kubectl get pods
Output:
netflix-abc123 0/1 CrashLoopBackOff 8
CrashLoopBackOff ante container start avvadaniki try chestundi, crash avtundi, Kubernetes malli restart chestundi.
Step 1 — Logs
kubectl logs netflix-abc123
Suppose:

Error: API_URL environment variable is missing
Ippudu root cause clear: application ki required environment variable missing.
Step 2 — Deployment configuration check
kubectl describe deployment netflix
Environment variables / ConfigMap references check cheyyali.
Suppose ConfigMap:

```
apiVersion: v1
kind: ConfigMap

metadata:
  name: netflix-config

data:
  API_URL: "http://backend-service"
```

Deployment lo ConfigMap inject cheyyali:

```
envFrom:
  - configMapRef:
      name: netflix-config
```

Full relevant section:

```
containers:
  - name: netflix
    image: mycompany/netflix:v2.3.0

    envFrom:
      - configMapRef:
          name: netflix-config

    ports:
      - containerPort: 80
```

Apply:
kubectl apply -f deployment.yaml
Then:
kubectl rollout status deployment/netflix
Check:
kubectl get pods
Logs:
kubectl logs <new-pod-name>
Healthy:
1/1 Running
Previous crashed container logs
Sometimes latest logs saripovu. App crash ayyi restart ayina previous instance logs:
kubectl logs netflix-abc123 --previous
Idi troubleshooting lo chala useful.

Task Example 8 — Service Traffic Pods ki Reach Avvatledu
Suppose Pods healthy:

```
netflix-abc   1/1 Running
netflix-def   1/1 Running
netflix-ghi   1/1 Running
```

Kani application access avvatledu.
Step 1 — Service check
kubectl get svc netflix-service
Then:
kubectl describe svc netflix-service
Suppose:
Selector: app=netflix
Endpoints: <none>
Pods running unna Service ki endpoints levu.
Step 2 — Pod labels check
kubectl get pods --show-labels
Suppose Pods:

```
netflix-abc   ...   app=netflix-web
```

But Service:

```
selector:
  app: netflix
```

Mismatch.
Service app=netflix Pods kosam search chestundi, but actual Pods app=netflix-web.
Step 3 — Correct selector

```
spec:
  selector:
    app: netflix-web
```

Apply:
kubectl apply -f service.yaml
Check:
kubectl describe svc netflix-service
Now endpoints appear:
Endpoints: 10.1.1.5:80,10.1.1.6:80,10.1.1.7:80
Appudu Service backend Pods ni identify chestundi.
This is a classic real-time issue: **Pods Running undadam alone sufficient kaadu; Service correct Pods ni select chestundo verify cheyyali.**

Task Example 9 — Ingress 502 Error
User:
https://netflix.example.com
open chesthe:
502 Bad Gateway
Step 1 — Ingress check
kubectl get ingress
Then:
kubectl describe ingress netflix-ingress
Check backend:
netflix-service:80
Step 2 — Service check
kubectl get svc netflix-service
kubectl describe svc netflix-service
Check:
Endpoints
Empty ayithe Service/Pod selector issue.
Endpoints unnayi ante next layer ki vellali.
Step 3 — Pod check
kubectl get pods -l app=netflix
All:
1/1 Running
unte Pods basic level lo healthy.
Step 4 — Application port check
Deployment:
containers:
- name: netflix
image: mycompany/netflix:v1
ports:
- containerPort: 80
Service:
ports:
- port: 80
targetPort: 80
targetPort actual application listening port tho match avvali.
Suppose application actually 8080 lo listen chestundi but Service:
targetPort: 80
unte traffic wrong port ki velthundi.
Correct:
ports:
- port: 80
targetPort: 8080
Apply:
kubectl apply -f service.yaml
Then test/verify.
Ingress 502 ki one fixed reason undadu. **Ingress → Service → Endpoints → Pod → Application port** chain lo exact failure point identify cheyyali.

Task Example 10 — Secret / Database Password Issue
Application database ki connect avvatledu.
Pod logs:
kubectl logs <pod-name>
Example:
Authentication failed for database user
App configuration Secret nunchi database credentials teesukuntundi anuko.
Secret check
kubectl get secret netflix-secret
Detailed metadata:
kubectl describe secret netflix-secret
Deployment Secret reference check:
kubectl describe deployment netflix

Suppose Deployment:

```
env:
 - name: DB_USERNAME
   valueFrom:
     secretKeyRef:
       name: netflix-secret
       key: DB_USERNAME

 - name: DB_PASSWORD
   valueFrom:
     secretKeyRef:
       name: netflix-secret
       key: DB_PASSWORD
```

Secret lo actual keys:

```
stringData:
  DB_USER: netflix
  DB_PASSWORD: example-password
```

Mismatch undi:
Deployment expects DB_USERNAME
Secret has DB_USER
Appudu correct key/reference align cheyyali.
Real production lo Secret values ni unnecessarily terminal output/Git repository lo expose cheyyakudadhu.
Fix tarvata:

```
kubectl apply -f secret.yaml
kubectl rollout restart deployment/netflix
kubectl rollout status deployment/netflix
```

Then:

```
kubectl get pods
kubectl logs <pod-name>
```

Real-Time lo Most Common Commands
Ee commands ni DevOps work lo frequently use chestav.
Resources chudataniki

```
kubectl get pods
kubectl get deployments
kubectl get replicasets
kubectl get services
kubectl get ingress
kubectl get configmaps
kubectl get secrets
kubectl get hpa
```

Specific namespace lo

```
kubectl get pods -n netflix-prod
```

Detailed information

```
kubectl describe pod <pod-name>
kubectl describe deployment netflix
kubectl describe service netflix-service
kubectl describe ingress netflix-ingress
kubectl describe hpa netflix-hpa
```

Logs

```
kubectl logs <pod-name>
```

Live logs:

```
kubectl logs -f <pod-name>
```

Previous crashed container:

```
kubectl logs <pod-name> --previous
```

YAML actual Kubernetes state chudataniki

```
kubectl get deployment netflix -o yaml
kubectl get service netflix-service -o yaml
```

Apply changes

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl apply -f ingress.yaml
```

Deployment rollout

```
kubectl rollout status deployment/netflix
```

History:

```
kubectl rollout history deployment/netflix
```

Rollback:

```
kubectl rollout undo deployment/netflix
```

Scaling

```
kubectl scale deployment netflix --replicas=6
```

Nodes

kubectl get nodes
Detailed:
kubectl describe node <node-name>
Resource usage
kubectl top pods
kubectl top nodes

Real-Time DevOps lo Main Troubleshooting Pattern
Any issue vachina immediate ga random YAML edit cheyyakunda ee order lo think cheyyi:
USER / ALERT
↓
What exactly is failing?
↓
Ingress
↓
Service
↓
Endpoints
↓
Pods
↓
Container
↓
Logs
↓
Application
↓
ConfigMap / Secret
↓
Image
↓
Node / Resources
Example:
502 error
↓
Ingress check
↓
Service check
↓
Endpoints check
↓
Pods check
↓
Pod logs
↓
Root cause identify
↓
Correct YAML/config/image
↓
kubectl apply
↓
rollout status
↓
Pods Ready
↓
Application test
Idi actual DevOps job lo important concept: company already YAML files correct ga unna, work mostly **new version deploy cheyyadam, replicas scale cheyyadam, production issues troubleshoot cheyyadam, configuration change cheyyadam, new environment create cheyyadam, HPA tune cheyyadam, networking/image/configuration problems fix cheyyadam, and every change tarvata verify cheyyadam**. YAML from scratch rayadam kanna existing system ni read chesi **"ee error vachindi → ee command run chesthe exact reason telustundi → aa reason batti ekkada change cheyyali → fix tarvata ela verify cheyyali"** ane skill real-time lo chala important